## 7. Hafta

# React III: API Tüketimi ve Yönlendirme

**◎ Öğrenme Hedefleri**

Bu bölümü tamamladığınızda:

- `fetch`/`Response`'u `async`/`await` ile kullanarak, bir isteğin `ok`/`status` alanlarıyla ağ hatası (Promise reddi) ile 4xx/5xx durum kodlarını (Promise'in normal çözülmesi) ayırt edebilecek; Hf.3'ün `Etkinlik`/`Koltuk`/`Bilet` tiplerini hiçbir dönüşüm yazmadan Hf.6'nın API cevaplarına uygulayan tiplenmiş bir `apiGet`/`apiPost` istemcisi oluşturabileceksiniz;
- `yukleniyor | hata | veri` üçlü durum modelini bir ayrımcı birleşim tipiyle tanımlayarak, bunu `useEffect` içinde veri çeken bir `useVeri<T>` özel hook'una uygulayabilecek; yarış durumunu (race condition) ve `AbortController` ile bir isteğin iptalini açıklayıp uygulayabileceksiniz;
- Tarayıcının aynı-köken politikasını, ön uçuş `OPTIONS` isteğini ve `Access-Control-Allow-Origin` başlığını açıklayarak, FastAPI `CORSMiddleware`'ini yalnızca belirli bir köken'e izin verecek biçimde yapılandırabilecek ve DevTools Network sekmesinde ön uçuş isteğini gözlemleyebileceksiniz;
- `react-router` ile `BrowserRouter`/`Routes`/`Route`/`Link`/`useParams`/`useNavigate`/ `Outlet` kullanarak parametreli rotalar ve ortak bir sayfa düzeni kurabilecek, tanımsız bir yola bir 404 sayfası gösterebileceksiniz;
- Bu becerileri birleştirip Etkinlik/Bilet Sisteminin ön yüzünü Hf.6'nın gerçek API'sine bağlayarak; etkinlik listesi, koltuk seçimi ve bilet formunu sunucudan gelen veriyle çalışır hâle getirip, koltuk çakışması (`409`) hatasını kullanıcıya gösteren, çok sayfalı bir SPA oluşturabileceksiniz.

Geçen hafta bir sunucu yazdınız, `curl` ile test ettiniz — ama React uygulamanız hâlâ Hf.3'ün sabit dizisini kullanıyordu, sunucunuzla hiç konuşmuyordu. Hf.1-5 yalnızca tarayıcıda yaşayan bir ön yüz kurdu, Hf.6 yalnızca kendi başına çalışan bir sunucu kurdu — bu iki parça şimdiye kadar hiç konuşmadı. Bu hafta bu iki parça birleşiyor: React uygulamanız artık `fetch` ile Hf.6'nın gerçek sunucusundan veri çekecek, yükleme/hata durumlarını yönetecek, tarayıcının "farklı adresten veri isteme" güvenlik kısıtına (CORS) çözüm bulacak ve tek-sayfalık uygulamanızı gerçek, adres çubuğunda görünen birden çok "sayfaya" (rota) bölecektir.

Dört ünite boyunca sırasıyla `fetch`/`Response` ile tiplenmiş bir API istemcisi yaz-

mayı (Ünite 1), bu isteklerin bekleme/hata durumlarını üç durumlu bir modelle yönetmeyi ve yarış durumunu `AbortController` ile önlemeyi (Ünite 2), tarayıcının aynı-köken güvenlik kısıtını (CORS) anlayıp FastAPI'de tek bir blokla çözmeyi (Ünite 3) ve son olarak `react-router` ile uygulamanızı gerçek, paylaşılabilir URL'lere sahip çok sayfalı bir yapıya bölmeyi (Ünite 4) göreceksiniz.

Bu haftanın da bilinçli sınırları vardır. Veritabanı/kalıcılık hâlâ yoktur (Hf.8); sunucu süreci yeniden başladığında veri hâlâ kaybolur — ön yüzünüz artık bu paylaşılan (ama hâlâ kalıcı olmayan) veriyi gerçek zamanlı gösterir. Kimlik doğrulama/oturum da yoktur (Hf.10); "Biletlerim" sayfası yalnızca bir URL sorgu parametresiyle basit bir liste gösterir. `react-router`'ın kendi veri çekme katmanı (`loader`, `createBrowserRouter`) bu hafta kullanılmaz — yalnızca bildirimsel API öğretilir. Sunucu tarafında bu hafta tek bir değişiklik yapılır: `app/main.py`'ye bir CORS bloğu eklenir — Hf.6'nın hiçbir rotası, modeli, durum kodu, camelCase kararı değişmez.

## 7.1 `fetch` ve Tiplenmiş İstemci

**1 • Ön-öğretim — Neden Önemli?**

Geçen hafta bir sunucu yazdınız, `curl` ile test ettiniz — ama React uygulamanız hâlâ Hf.3'ün sabit dizisini kullanıyor, sunucunuzdan habersiz. `curl`'ün yaptığı şeyi (bir HTTP isteği gönderip cevabı okuma) tarayıcı içindeki JavaScript kodunuzun da yapabilmesi gerekiyor — bu ünite bunun standart yolunu tanıtır.

**1 • Ön-öğretim — Ön Bilgi**

Hf.3 Ünite 4'ün `Promise`/`async`/`await` bilgisini hatırlayın — `fetch` de birden yeni bir Promise kaynağıdır, sözdizimi aynen kullanılır. Hf.6'nın tüm rota/durum kodu/camelCase kararları da bu ünitenin ön bilgisidir — bu haftanın `fetch`'i O sözleşmeyi, değiştirmeden, tüketir.

Hf.1'de yalnız kavram düzeyinde gördüğünüz istemci-sunucu modeli bu ünitede kendi yazdığınız TypeScript kodunda somutlaşıyor. Sırasıyla `fetch`'in döndürdüğü `Response` nesnesini, bu haftanın en kritik ayrımını (ağ hatası ile bir HTTP hata cevabı arasındaki fark) ve bu ayrımı tek bir yerde toplayan tiplenmiş bir istemci katmanını göreceksiniz.

### 7.1.1 `fetch`: tarayıcının HTTP istemcisi

**fetch**: Bir HTTP isteği başlatan, bir `Promise<Response>` döndüren tarayıcı API'si (*fetch API*).

Tarayıcı, `curl`'ün yaptığı işi yapan bir fonksiyonu zaten içerir: **fetch**. `fetch(url)` çağrısı Hf.3'te öğrendiğiniz gibi bir `Promise` döndürür — bu Promise'in içindeki değer bir `Response` nesnesidir. `await` ile bu Promise'in çözülmesini beklersiniz:

```
const response = await fetch('http://127.0.0.1:8000/etkinlikler');
```

Bu satır çalıştığında istek henüz tamamlanmamış olabilir — `await` tam olarak Hf.3'te gördüğünüz gibi, cevap gelene kadar fonksiyonun bu noktada duraklamasını sağlar.

### 7.1.2 `Response`: durum kodu ve gövde iki ayrı adımdır

`response` nesnesi iki önemli alan/metot taşır: `.ok` (durum kodu 200-299 aralığındaysa `true`), `.status` (sayısal durum kodu, ör. `404`) ve `.json()` — gövdeyi JSON

olarak ayrıştıran, kendisi de bir Promise döndüren bir metot. Bu son nokta sıkça unutulur: gövdeyi okumak ikinci, ayrı bir `await` adımı gerektirir:

```
const response = await fetch(url); // 1. adim: cevap geldi
const veri = await response.json(); // 2. adim: govde ayristirildi
```

**Ortak Hata / Tuzak**

Yalnız `const veri = await fetch(url);` yazıp doğrudan `veri.baslik` gibi bir alana erişmeye çalışmak yaygın bir hatadır — `fetch`'in döndürdüğü şey bir `Response` nesnesidir, JSON verisinin kendisi değildir; gövdeye erişmek için mutlaka ikinci bir `await response.json()` adımı gerekir.

### 7.1.3 En kritik ayrım: ağ hatası mı, HTTP hata cevabı mı?

Bu ünitenin merkezindeki soru şudur: `fetch`'in döndürdüğü Promise ne zaman reddedilir (reject), ne zaman çözülür (resolve)? Cevap sezgilere aykırı görünebilir: `fetch` yalnızca bir **ağ hatası** durumunda reddedilir. Sunucu bir `404`, `422` ya da `500` dönse bile bu bir ağ hatası değildir — sunucu bir cevap üretmiştir, Promise normal şekilde çözülür; yalnızca `response.ok` `false` olur, `response.status` ilgili hata koduna eşit olur.

**ağ hatası**: Sunucuya hiç ulaşılamadığı için `fetch`'in Promise'ini reddeden hata türü (*network error*); DNS çözülemedi, bağlantı hiç kurulamadı ya da CORS isteği tamamen engelledi.

**Analoji**

Bunu bir telefonla sipariş vermeye benzetin: numarayı çevirirsiniz (istek); ya karşı taraf hiç açmaz (ağ hatası — arama başarısız sayılır), ya da biri açar ve "siparişiniz alındı" (200) ya da "üzgünüz, o ürün bende yok" (404) der — ikinci durumda görüşme başarıyla tamamlanmıştır, yalnızca söylenen cevap olumsuzdur. Telefonu kapatmadan önce karşı tarafın ne söylediğini (durum kodunu) dinlemezseniz ret cevabını kaçırırsınız.

Şekil 7.1 bu iki karar noktasını ("sunucuya ulaşıldı mı?" ve "`response.ok` mı?") ayrı ayrı gösterir — sol dal (ağ hatası) yalnız DNS/bağlantı/CORS engelinde tetiklenir, sağ dal ise her HTTP cevabından (2xx dahil) geçer.

**Ortak Hata / Tuzak**

Öğrencilerin en sık düştüğü yanlış anlama şudur:

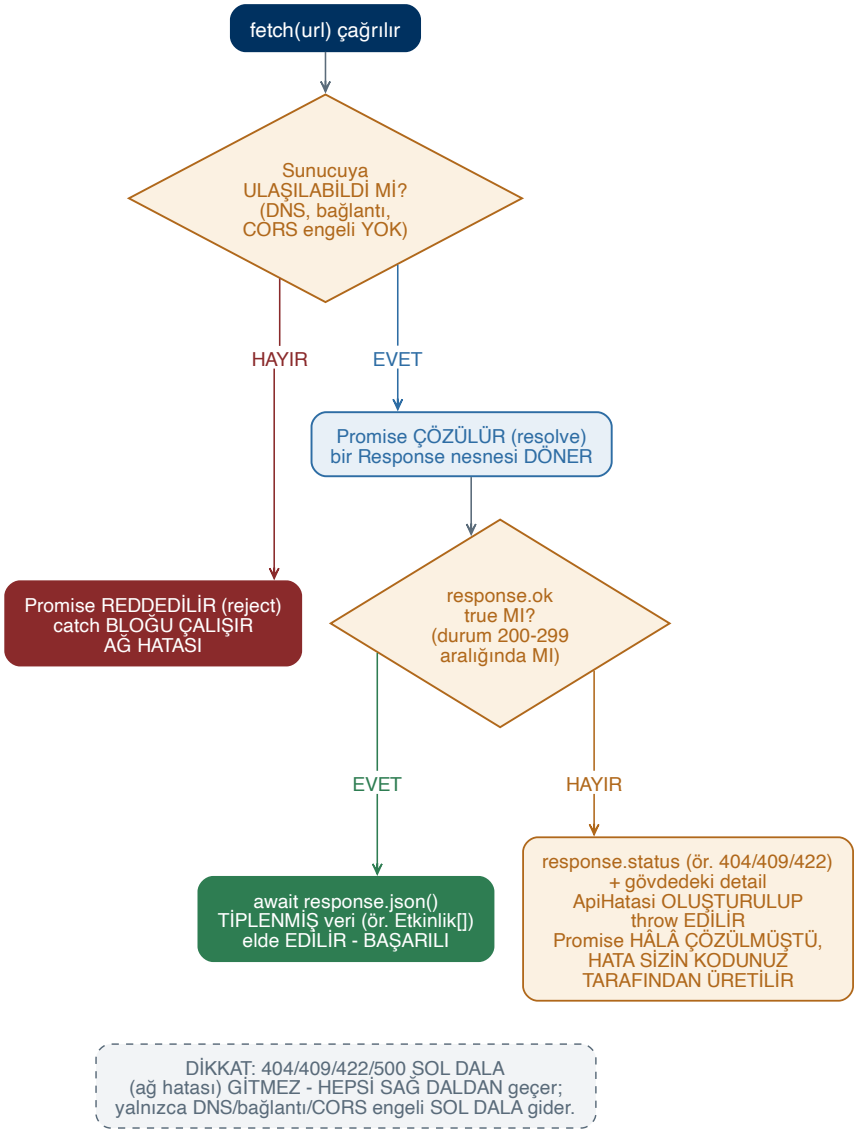
```
try {
  const response = await fetch(url);
  // burada 404 YAKALANMAZ!
} catch {
  // yalnizca AG hatasinda calisir
}
```

`response.ok` kontrol edilmeden bir `404`/`409` sessizce "başarılı" gibi işlenir — `catch` bloğu hiç çalışmaz, çünkü Promise zaten normal şekilde çözülmüştür. Durum kodunu kontrol etmek, `try`/`catch` yazmaktan ayrı, ek bir adımdır.

### 7.1.4 Tiplenmiş sarmalayıcı fonksiyonlar: `apiGet`/`apiPost`

Bu ünite, HTTP isteği ve hata kontrolünü tek bir yerde toplayan bir **tiplenmiş istemci** kurar. Bu katman olmadan, her bileşende `fetch` + `response.ok` kontrolü +

**tiplenmiş istemci**: HTTP isteği ve hata kontrolünü tek yerde toplayan, jenerik tip alan yardımcı fonksiyon (*typed client*).




Şekil 7.1: Bir `fetch` çağrısının ağ hatasında (Promise reddi) ile sunucunun herhangi bir HTTP cevabında (Promise'in normal çözülmesi) izlediği ayrı yollar.

`response.json()` üçlüsünü tekrar tekrar yazmanız gerekirdi — bu tekrar yalnızca ek yazım yükü değil, hata kontrolünün bir bileşende unutulma riskidir de. Tiplenmiş istemci bu üç adımı tek bir dosyada toplayarak her çağrının aynı kurallara uymasını garanti eder. `response.ok` `false` olduğunda bu fonksiyonlar, gövdedeki hata detayını taşıyan özel bir hata sınıfı fırlatır — bu sayede çağıran kod `try/catch` ile tek bir hata türünü yakalamayı öğrenir. Önce taban URL sabitine ve hata sınıfının tanımına bakın:

```
export const TABAN_URL = 'http://127.0.0.1:8000';
```

```
export class ApiHatasi extends Error {
  status: number;
  detail: unknown;

  constructor(status: number, detail: unknown) {
    super(`API hatası: ${status}`);
    this.name = 'ApiHatasi';
    this.status = status;
    this.detail = detail;
  }
}
```

`TABAN_URL`'in tek bir sabitte tutulması, her `fetch` çağrısının sunucu adresini tekrar tekrar yazmamasını sağlar.

> **İpucu**
>
> Gerçek projelerde bu adres genelde bir `.env` dosyasından okunur — ortam değişkeni ile bu tür yapılandırma Hf.12'nin konusudur, bu hafta tek bir sabit yeterlidir.

`apiGet`/`apiPost`/`apiPatch`/`apiDelete` fonksiyonları bu iki parça üzerine kurulur — her biri, `cevabiIsle` adlı ortak bir yardımcı fonksiyon aracılığıyla `response.ok` kontrolünü ve `ApiHatasi` fırlatmasını paylaşır:

```
export async function apiGet<T>(
  yol: string,
  { signal }: IstekSecenekleri = {},
): Promise<T> {
  const response = await fetch(`${TABAN_URL}${yol}`, { signal });
  return cevabiIsle<T>(response);
}
```

```
export async function apiPost<TIn, TOut>(
  yol: string,
  govde: TIn,
  { signal }: IstekSecenekleri = {},
): Promise<TOut> {
  const response = await fetch(`${TABAN_URL}${yol}`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(govde),
    signal,
  });
  return cevabiIsle<TOut>(response);
}
```

`apiGet<T>` jenerik bir tip parametresi (`T`) alır — bu, Hf.3'ün `Liste<T>` kalıbının bu haftaki doğrudan karşılığıdır: fonksiyonun ne döneceğini kullanım anında siz belirlersiniz (`apiGet<Etkinlik[]>(...)` gibi).

### 7.1.5 camelCase kararının bugün Ödediği bedel

Hf.6'da alınan "JSON alanları her zaman camelCase" kararını hatırlayın. Bu kararın getirisi bugün somutlaşıyor: `apiGet<Etkinlik[]>` fonksiyonunun `'/etkinlikler'` uç noktasına yaptığı çağrının döndürdüğü veri, Hf.3'ün `interface Etkinlik` tanımına hiçbir dönüşüm kodu yazmadan doğrudan uyar — sunucu zaten camelCase JSON döndüğü için hiçbir `snakeToCamel(...)` fonksiyonu yazmanız gerekmez. İki farklı dilde (TypeScript/Python) tanımlanmış iki modelin, Hf.6'nın OpenAPI şemasında nasıl buluştuğunun somut kanıtı budur.

Bu deseni her varlık için ayrı, küçük bir modülde toplarsınız; `src/api/` klasörü, Hf.6'nın "her varlık kendi dosyasında" disiplininin ön yüz karşılığıdır:

```
export function etkinlikleriGetir(
  mekan?: string,
  signal?: AbortSignal,
): Promise<Etkinlik[]> {
  const sorgu = mekan ? `?mekan=${encodeURIComponent(mekan)}` : '';
  return apiGet<Etkinlik[]>(`/etkinlikler${sorgu}`, { signal });
}

export function etkinlikGetir(id: number, signal?: AbortSignal): Promise<
    Etkinlik> {
  return apiGet<Etkinlik>(`/etkinlikler/${id}`, { signal });
}
```

```
export function koltuklariGetir(
  etkinlikId: number,
  durum?: KoltukDurumu,
  signal?: AbortSignal,
): Promise<Koltuk[]> {
  const sorgu = durum ? `?durum=${durum}` : '';
  return apiGet<Koltuk[]>(`/etkinlikler/${etkinlikId}/koltuklar${sorgu}`, {
    signal,
  });
}

export function koltukGuncelle(id: number, durum: KoltukDurumu): Promise<
    Koltuk> {
  return apiPatch<{ durum: KoltukDurumu }, Koltuk>(`/koltuklar/${id}`, {
      durum });
}
```

```
export interface BiletOlusturGirdisi {
  koltukId: number;
  kullaniciAdi: string;
}

export function biletOlustur(girdi: BiletOlusturGirdisi): Promise<Bilet> {
  return apiPost<BiletOlusturGirdisi, Bilet>('/biletler', girdi);
}

export function biletleriGetir(
  kullaniciAdi: string,
  signal?: AbortSignal,
): Promise<Bilet[]> {
  return apiGet<Bilet[]>(
    `/biletler?kullaniciAdi=${encodeURIComponent(kullaniciAdi)}`,
    { signal },
```

```
  );
}
```

Hf.5'ten beri kullandığınız `veri.ts`'in sabit (mock) diziler, bu haftadan itibaren silinir — artık her veri, yukarıdaki üç modül üzerinden, gerçek sunucudan gelir.

**3 • Katılım — Şimdi Siz Deneyin**

Sunucu çalışırken tarayıcı konsolunda `apiGet<Etkinlik[]>` fonksiyonunu `'/etkinlikler'` ile çağırıp sonucu yazdırın; ardından aynı fonksiyonu (bu kez `apiGet<Etkinlik>` olarak) var olmayan bir id ile, `'/etkinlikler/999'` biçiminde çağırıp ne olduğunu — hatanın fırlatılıp fırlatılmadığını, `status`/`detail` alanlarını — gözlemleyin.

**Geribildirim.** İlk çağrı Hf.6'nın canlı etkinlik listesini — Hf.3'ün sabit dizisiyle aynı şekilde, hiçbir alan dönüşümü olmadan — döner. İkinci çağrı `apiGet`'in içindeki `response.ok` kontrolü sayesinde bir `ApiHatasi` (`status: 404`) fırlatır — `fetch`'in kendisi bu isteği reddetmemişti; `apiGet` bunu sizin için bir hataya çevirdi.

**Kendinizi Sınayın 7.1**

`fetch('/etkinlikler/999')` sunucudan `404` cevabı döndüğünde, bu isteğin Promise'i reddedilir (reject) mi, yoksa normal şekilde çözülür (resolve) mü? `response.ok` bu durumda ne gösterir?

**5 • İzleme — Aktarım — Başka Nerede?**

Bugün kurduğunuz `apiGet`/`apiPost`/`ApiHatasi` deseni, dönem boyunca her yeni API çağrısında (Hf.10'un giriş isteği, Hf.13'ün WebSocket öncesi ilk bağlantı isteği) aynen yeniden kullanılacak — yeni bir varlık için yalnızca `src/api/` altına bir dosya daha ekleyeceksiniz, `istemci.ts`'in kendisi değişmeyecek.

## 7.2 Yükleme, Hata ve Boş Durumları

**1 • Ön-öğretim — Neden Önemli?**

Ünite 1'in `apiGet` çağrısı anında bir sonuç dönmez — ağ üzerinden bir istek gider, bir süre (milisaniyeler de olsa) beklenir. Bu süre boyunca ekranda hiçbir şey göstermezseniz kullanıcı sayfanın donduğunu/bozulduğunu düşünür; istek başarısız olduğunda sessizce hiçbir şey yapmazsanız kullanıcı neden hiçbir veri görmediğini hiç anlamaz.

**1 • Ön-öğretim — Ön Bilgi**

Hf.5 Ünite 1'in `useEffect`+bağımlılık dizisi+temizleme fonksiyonu bilgisini hatırlayın — ve o haftanın bilinçli olarak yarım bıraktığı cümleyi: "Hf.7'de bir API'den veri çekerken de aynı hook'u kullanacaksınız." Bu ünitede bu cümle gerçekleşiyor. Hf.5 Ünite 4'ün özel hook kalıbını (`useSepet`, `useGeriSayim`) da

hatırlayın — bu ünitedeki `useVeri` aynı kalıbın yeni bir örneğidir.

Ünite 1 `fetch`'in ne zaman ağ hatası verdiğini, ne zaman bir HTTP hata cevabı ürettiğini ayırt etmeyi öğretti. Bu ünite bir adım ileri gider: bu bekleme süresini ve olası sonuçları (başarı/hata) ekranda nasıl göstereceğinizi, bunu tek bir yerde toplayan bir hook'a nasıl dönüştüreceğinizi ve iki istek yarıştığında (race condition) ortaya çıkan sinsi bir hatayı nasıl önleyeceğinizi gösterir.

### 7.2.1 Üç durumlu veri modeli

Bir ekranın bir API çağrısı sırasında alabileceği durumları tek bir tipte toplayan bir **ayrımcı birleşim tipi** tanımlarsınız:

**ayrımcı birleşim tipi**: Ortak bir alanla (`tur`) hangi ek alanların[1] geçerli olduğunu[2] belirten union tipi[3] (*discriminated union*).[4]

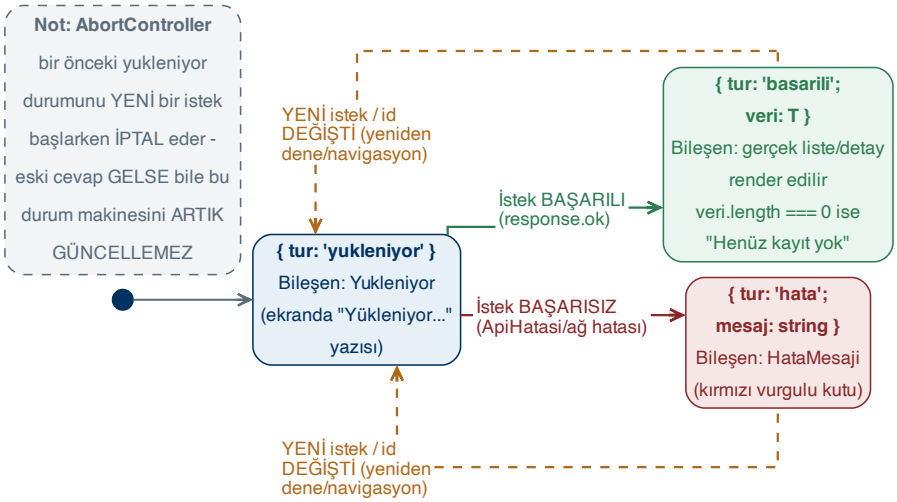
```
export type VeriDurumu<T> =
  | { tur: 'yukleniyor' }
  | { tur: 'hata'; mesaj: string }
  | { tur: 'basarili'; veri: T };
```

Bu, Hf.3 Ünite 3'ün union/literal tip bilgisinin (`'bos' | 'secili' | 'dolu'`) bu kez ortak bir `tur` alanıyla ayırt edilen, her dalın farklı ek alanlar taşıdığı genişletilmiş bir biçimidir. TypeScript `durum.tur === 'basarili'` kontrolünden sonra `durum.veri`'ye erişimin güvenli olduğunu bilir — `durum.tur === 'yukleniyor'` iken `durum.veri`'ye erişmeye çalışmak derleme hatası verir; bu tip daraltma davranışı Hf.3'ten tanıdıktır.

**Analoji**

Bunu bir kargo takip ekranına benzetin: paket "Hazırlanıyor" (yükleniyor), "Teslim edilemedi" (hata) ya da "Teslim edildi, içerik burada" (veri) diyebilir — ekran her zaman bu üç durumdan birini gösterir, asla boş/belirsiz kalmaz.

Şekil 7.2, bu üç durumun hangi bileşenle eşlendiğini bir durum makinesi olarak gösterir.




Şekil 7.2: `yukleniyor`/`hata`/`basarili` durumları arasındaki geçişler ve her durumun render ettiği bileşen.

### 7.2.2 `useEffect` içinde veri çekme

Hf.5'in yarım bıraktığı cümle burada tamamlanır: veri çekmek de bir yan etkidir, bu yüzden `useEffect` içinde yapılır. Ama bir tuzak vardır — `useEffect`'in kendisi bir Promise döndüremez, bu yüzden `async` bir fonksiyon doğrudan efekte verilemez. Çözüm, efekt içinde ayrı bir `async` fonksiyon tanımlayıp hemen çağırmaktır:

```
useEffect(() => {
  async function veriyiGetir() {
    const veri = await apiGet<Etkinlik[]>('/etkinlikler');
    setDurum({ tur: 'basarili', veri });
  }
  veriyiGetir();
}, []);
```

**Ortak Hata / Tuzak**

`useEffect(async () => { ... }, [])` yazmak — efekt fonksiyonunun kendisini `async` yapmak — React'te bir uyarı/hata kaynağıdır; React efektin dönüş değerini bir temizleme fonksiyonu olarak bekler, bir Promise değil. Efekt içinde ayrı bir `async` fonksiyon tanımlayıp onu çağırmak bu sorunu ortadan kaldırır.

### 7.2.3 Yarış durumu: iki istek aynı anda Yola Çıkarsa

Bir kullanıcı `/etkinlikler/3`'ten `/etkinlikler/7`'ye hızlıca geçtiğinde iki ayrı istek yola çıkar. Ağ gecikmeleri yüzünden 3 numaralı etkinliğin cevabı 7 numaralınınkinden daha geç gelebilir — bu durumda ekranda 7 numaralı etkinliğin sayfasında yanlışlıkla 3 numaralının verisi görünebilir; bir **yarış durumu** oluşur — "eski cevap yeni cevabı ezer".

**yarış durumu**: Birden fazla isteğin cevaplarının beklenmedik sırayla gelip ekranı yanlış güncellemesi (*race condition*).

**Analoji**

Bunu iki ayrı gişeye art arda gönderdiğiniz iki mektuba benzetin: ikinci mektubun cevabı, birincisinin cevabından önce gelirse, sonra gelen birinci cevap masanızdaki güncel bilgiyi yanlışlıkla üzerine yazar; bir kurye çağırıp "ilk mektubu artık yola çıkarma" demek (iptal) bu karışıklığı önler.

### 7.2.4 `AbortController` ile isteği iptal etme

Bu "kuryeyi durdurma" işini tarayıcı yerleşik bir mekanizmayla sunar: **İstek iptali**. `useEffect` İçinde bir `AbortController` oluşturulup `controller.signal`'ın `apiGet`'e (isteğe bağlı `{ signal }` parametresi olarak) geçirilmesi gerekir; efektin temizleme fonksiyonunda (Hf.5 Ünite 1'den tanıdık) `controller.abort()` çağrılır. Bu sayede bağımlılık değiştiğinde (ör. yeni bir `id`) ya da bileşen sahneden kalktığında (unmount) önceki istek iptal edilir — cevabı gelse bile `AbortError` fırlatıldığı için state güncellenmez.

**İstek iptali**: Tamamlanmadan önce bir ağ isteğini durdurma mekanizması (*request cancellation*).

### 7.2.5 `useVeri<T>`: hepsi tek bir hook'ta

Hf.5 Ünite 4'ün özel hook kalıbının yeni bir örneği olarak, yukarıdaki üç parçayı (`useEffect` + `useState<VeriDurumu<T>>` + `AbortController`) tek bir yerde toplarsınız; aşağıdaki üç kod parçası bu hook'un gövdesini adım adım gösterir. İlk parça

isteği başlatır: efekt her çalıştığında yeni bir `AbortController` oluşturulur, durum `yukleniyor`'a çekilir ve `cagri` çağrılıp cevap başarıyla gelirse `basarili` durumuna geçilir:

```
export function useVeri<T>(
  cagri: (signal: AbortSignal) => Promise<T>,
  bagimliliklar: unknown[],
): VeriDurumu<T> {
  const [durum, setDurum] = useState<VeriDurumu<T>>({ tur: 'yukleniyor' });

  useEffect(() => {
    const controller = new AbortController();
    setDurum({ tur: 'yukleniyor' });

    async function veriyiGetir() {
      try {
        const veri = await cagri(controller.signal);
        setDurum({ tur: 'basarili', veri });
```

`cagri` parametresi, `AbortSignal`'i alan ve bir `Promise<T>` döndüren bir fonksiyondur — bu sayede `useVeri` hangi API çağrısının yapılacağını kendisi bilmez, yalnızca çağrının nasıl yönetileceğini (yükleniyor/hata/başarılı, iptal) bilir.

İkinci parça hatayı ele alır: `AbortError` sessizce (state güncellenmeden) yutulur — bu, iptal edilen bir isteğin ekranda yanlışlıkla hata göstermesini önler; diğer tüm hatalarda `mesaj` değişkeni, hatanın `ApiHatasi` (sunucudan gelen `detail` alanı), sıradan bir `Error` ya da bilinmeyen bir değer olmasına göre kullanıcıya gösterilecek tek bir metne indirgenir:

```
if (hata instanceof DOMException && hata.name === 'AbortError') {
  return;
}
const mesaj =
  hata instanceof ApiHatasi
    ? typeof hata.detail === 'string'
      ? hata.detail
      : hata.message
    : hata instanceof Error
      ? hata.message
      : 'Bilinmeyen bir hata oluştu.';
setDurum({ tur: 'hata', mesaj });
```

Üçüncü ve son parça `veriyiGetir`'i çağırır ve efektin temizleme fonksiyonunda `controller.abort()` ile önceki isteği iptal eder:

```
    veriyiGetir();

    return () => controller.abort();
  }, bagimliliklar);

  return durum;
}
```

Her sayfa bileşeni bu üç adımı tekrar tekrar yazmak yerine yalnızca tek satır yazar:

```
const durum = useVeri<Etkinlik[]>(
  (signal) => etkinlikleriGetir(undefined, signal),
  [],
);
```

### 7.2.6 Yükleme/hata bileşenleri ve boş liste durumu

`durum.tur`'a göre hangi bileşenin render edileceğine karar veren bir koşullu render deseni (Hf.4 Ünite 4'ten tanıdık) kurulur:

```
export function Yukleniyor() {
  return <p className="sayfa-yukleniyor">Yükleniyor…</p>;
}
```

```
interface HataMesajiProps {
  mesaj: string;
}

export function HataMesaji({ mesaj }: HataMesajiProps) {
  return <p className="sayfa-hata">{mesaj}</p>;
}
```

Bu iki bileşen ile Ünite 1'in `etkinlikleriGetir` fonksiyonu bir araya geldiğinde `EtkinliklerSayfasi` şu hâle gelir:

```
export function EtkinliklerSayfasi() {
  const durum = useVeri<Etkinlik[]>(
    (signal) => etkinlikleriGetir(undefined, signal),
    [],
  );

  if (durum.tur === 'yukleniyor') {
    return <Yukleniyor />;
  }
  if (durum.tur === 'hata') {
    return <HataMesaji mesaj={durum.mesaj} />;
  }

  return (
    <section className="bolum">
      <h2>Etkinlikler</h2>
      <EtkinlikListesi etkinlikler={durum.veri} />
    </section>
  );
}
```

Boş liste durumu (`durum.tur === 'basarili' && durum.veri.length === 0`) ayrı bir kısa mesajla ("Henüz etkinlik yok.") ele alınır — bu mantık `EtkinlikListesi`'nin (Hf.4'ten aynen) kendi içindedir.

### 7.2.7 İyimser güncelleme: fikir düzeyinde bir önizleme

**iyimser güncelleme**: Sunucu cevabını beklemeden arayüzü önce güncelleyip, hata gelirse geri alma deseni (*optimistic update*).

Bir koltuğa tıklandığında, sunucunun `PATCH /koltuklar/{id}` cevabını beklemeden önce arayüzün koltuğu hemen "seçili" göstermesine **iyimser güncelleme** denir — kullanıcı deneyimi hızlı hisseder. Sunucu cevabı `409` ile dönerse (koltuk bu arada başkası tarafından alınmışsa) arayüzün bu değişikliği geri alması ve bir hata mesajı göstermesi gerekir.

**Analoji**

Bunu bir garsonun siparişinizi alır almaz masanıza "hazırlanıyor" fişini bırakmasına benzetin: mutfaktan "üzgünüz, o malzeme bitti" cevabı gelirse fiş geri alınır ve size haber verilir — ama siz o ana kadar siparişin alındığını anında

görürsünüz, mutfağın cevabını beklemek zorunda kalmazsınız.

Bu ünitede yalnızca fikri tanıdınız; `secilenKoltuklar` ile `koltukGuncelle`'nin birlikte kullanıldığı tam kod, Ünite 4'ün `EtkinlikDetaySayfasi` bölümünde gösterilecektir.

**3 • Katılım — Şimdi Siz Deneyin**

`EtkinlikDetaySayfasi`'nda arka arkaya farklı etkinlik bağlantılarına hızlıca tıklayıp konsola basılan `id` değerleriyle ekranda görünen etkinliğin tutarlı olup olmadığını gözlemleyin; sonra `useVeri` içindeki `AbortController`'ı geçici olarak devre dışı bırakıp aynı deneyi tekrar yapın.

**Geribildirim.** `AbortController` aktifken ekran her zaman son tıklanan etkinliği gösterir — önceki istekler iptal edildiği için state'i güncellemez. Devre dışı bırakıldığında ağ gecikmesine bağlı olarak ekranda yanlış (eski) bir etkinliğin kısa süreliğine görünmesi mümkün olur — yarış durumunun somut kanıtıdır.

**Kendinizi Sınayın 7.2**

`useVeri` hook'unda `AbortController` kullanmasaydınız, kullanıcı `/etkinlikler/3`'ten `/etkinlikler/7`'ye hızlıca geçtiğinde hangi sırayla gelen cevaplar ekranı yanlış gösterebilirdi?

**5 • İzleme — Aktarım — Başka Nerede?**

Bu üç durumlu model (`yukleniyor`/`hata`/`veri`) Hf.10'un giriş formunda ("giriş yapılıyor..."/"şifre yanlış"/"başarılı"), Hf.13'ün WebSocket bağlantı durumunda ("bağlanıyor..."/"bağlantı koptu"/"bağlı") aynı iskeletle tekrar kullanılacak.

## 7.3 CORS İlk Temas

**1 • Ön-öğretim — Neden Önemli?**

Geçen hafta bırakılan cümle gerçekleşiyor: ilk `fetch` denemenizde tarayıcı konsolu kırmızı bir "CORS policy" hatasıyla dolar — isteğiniz doğru yazılmış olsa bile tarayıcı cevabı sizden saklar. Bu tarayıcının bir güvenlik önlemidir, bir hata değildir — anlaşılmadan çözülemez.

**1 • Ön-öğretim — Ön Bilgi**

Hf.1 Ünite 4'ün DevTools Network sekmesi deneyimini hatırlayın — bu ünitede aynı sekmeyi yeni bir tür isteği (`OPTIONS`) gözlemlemek için kullanacaksınız. Hf.6'nın `app/main.py` dosyasını da hatırlayın — bu hafta yalnızca bir blok eklenecek, geri kalanı dokunulmaz.

Ünite 1-2 `fetch`'i ve durum yönetimini kurdunuz; ama Vite'ın geliştirme sunucusu (`5173`) üzerinden FastAPI'nize (`8000`) gerçek bir istek attığınızda tarayıcı bu

isteği varsayılan olarak engeller. Bu ünite bu engelin neden var olduğunu, tarayıcının bu engeli nasıl uyguladığını ve sunucu tarafında tek bir yapılandırma bloğuyla bunu nasıl aşacağınızı gösterir.

### 7.3.1 Köken (origin): bir web adresinin "kimliği"

**köken**: Protokol+alan adı+port üçlüsüyle tanımlanan kaynak kimliği (*origin*).

Bir **köken**, protokol + alan adı + port üçlüsüdür. `http://localhost:5173` ile `http://127.0.0.1:8000` farklı iki köken'dir — yalnızca port farkı bile yeterlidir. Vite'ın geliştirme sunucusu `5173` portunda, FastAPI `8000` portunda çalışır — bu haftadan itibaren her `npm run dev` oturumu bu iki portu aynı anda açık tutmayı gerektirir.

### 7.3.2 Aynı-köken politikası: tarayıcının varsayılan kapıcısı

**aynı-köken politikası**: Tarayıcının farklı origin'e giden isteklerin cevabını script'e kapatan varsayılan kuralı (*same-origin policy*).

Tarayıcının varsayılan davranışı, bir sayfanın (`5173`) kendi köken'i dışındaki bir adrese (`8000`) yaptığı `fetch` isteğinin cevabını JavaScript koduna okutturmamasıdır — bir **aynı-köken politikası**. Bu bir güvenlik önlemidir: bir kötü niyetli sitenin, kullanıcının giriş yapmış olduğu bankası gibi başka bir siteye sizin adınıza istek atıp cevabı okumasını engeller. Bu güvenlik boyutunun derinlemesine işlenmesi Hf.11'in konusudur, burada yalnızca "neden var" sorusuna kısaca değinilir.

**Analoji**

Bunu bir apartmanın kapıcısının varsayılan olarak yalnızca kendi apartmanından gelen taleplere kapı açmasına benzetin: başka bir binadan (farklı köken) gelen bir kuryeyi, siz izin vermediğiniz sürece içeri almaz — bu, kötü niyetli bir binanın sizin adınıza komşu apartmana (banka sitesi gibi) istek gönderip cevabı okumasını engeller.

**Ortak Hata / Tuzak**

Bu engeli ilk gördüğünüzde isteğin sunucuya hiç ulaşmadığını düşünebilirsiniz — oysa çoğu zaman istek sunucuda gerçekten çalışmıştır (Network sekmesinde durum kodu görünebilir); tarayıcı yalnızca cevabı JavaScript koduna iletmez. Sorun sunucunun işini yapmaması değil, tarayıcının cevabı saklamasıdır.

### 7.3.3 Ön uçuş isteği: izin sormak için gönderilen `OPTIONS`

**ön uçuş isteği**: Basit olmayan bir isteğin gönderilmeden önce izin sormak için yollanan `OPTIONS` isteği (*preflight request*).

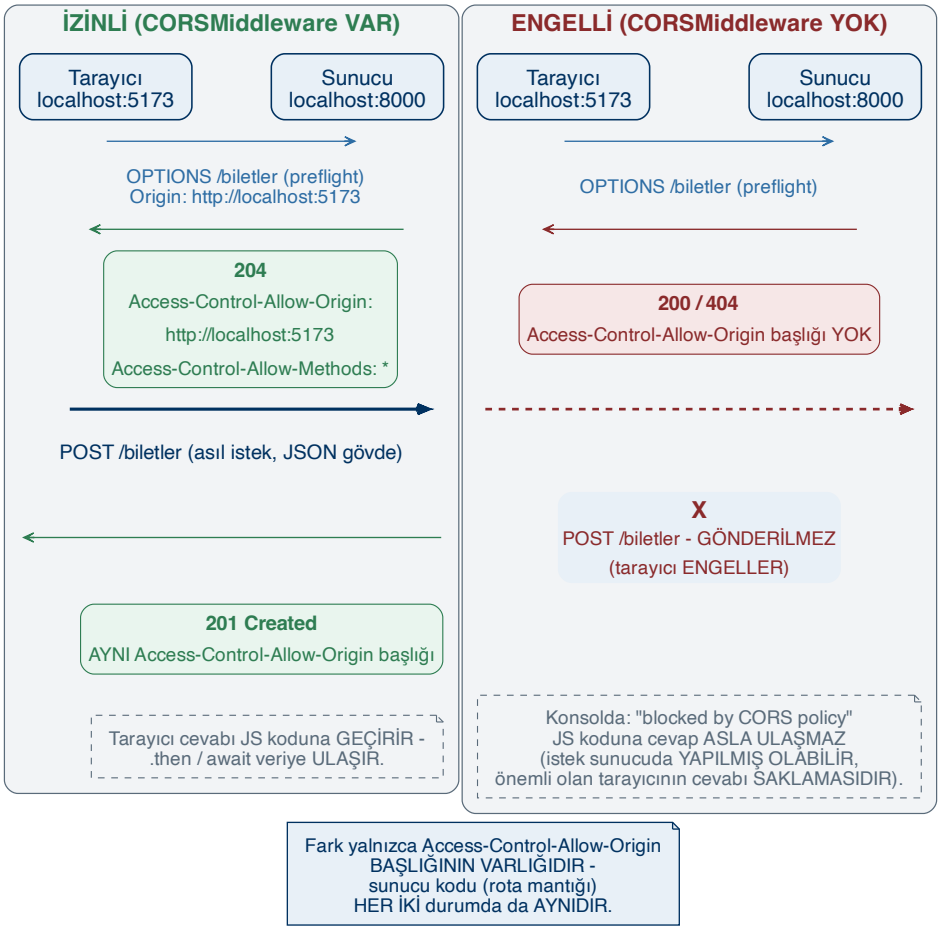
`Content-Type: application/json` gövdeli bir `POST`/`PUT`/ `PATCH`/`DELETE` isteği gibi "basit olmayan" istekler için, tarayıcı asıl isteği göndermeden önce otomatik olarak bir **ön uçuş isteği** gönderir: "ben bu köken'den, bu metotla, bu başlıklarla istek atmak istiyorum, izin var mı?" diye sorar. Sunucu uygun `Access-Control-Allow-*` başlıklarıyla cevap verirse tarayıcı asıl isteği gönderir; vermezse asıl istek hiç gönderilmez.

**Analoji**

Bunu bir kuryenin kapıya gelmeden önce, apartman yönetimine bir not bırakıp "böyle bir paket getirebilir miyim?" diye sormasına benzetin: yönetim (sunucu) "evet, bu bina adına (köken) izin var" diye cevap verirse kurye asıl paketi

getirir; cevap gelmez/olumsuz gelirse paket hiç yola çıkmaz.

Şekil 7.3, izinli bir akış ile CORSMiddleware olmadan engellenen bir akışı yan yana gösterir.




Şekil 7.3: 5173'ten 8000'e giden bir istekte ön uçuş `OPTIONS` + izinli başlıklarla tamamlanan akış ile başlık eksikliğinde engellenen akışın karşılaştırması.

### 7.3.4 İzinli köken başlığı ve `CORSMiddleware`

**izinli köken başlığı**: Sunucunun hangi origin'lere cevabı okuma izni verdiğini bildiren HTTP başlığı (*Access-Control-Allow-Origin*).

Sunucunun cevabına eklediği, **izinli köken başlığı**, "bu cevabı hangi köken'lerin JavaScript'inin okuyabileceğini" belirtir. Tarayıcı bu başlığı kontrol eder, isteği yapan sayfanın köken'iyle eşleşmiyorsa cevabı JS koduna ulaştırmaz — istek sunucuda çalışmış olabilir, ama tarayıcı sonucu saklar.

FastAPI bu başlığı elle yazmanızı gerektirmez; hazır bir ara katman (*middleware*) sunar. Bu haftanın `app/main.py`'ye eklenen tek bloğu şudur:

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

`allow_origins` listesinin yalnızca geliştirme adresini — `localhost:5173` — içermesine dikkat edin.

**Analoji**

`allow_origins` listesini apartman yönetiminin kapıcıya verdiği yazılı izinli ziyaretçi listesine benzetin: listede olmayan bir bina adına gelen hiçbir paket içeri alınmaz, listedeki tek bir bina (`localhost:5173`) her zaman içeri alınır.

**Dikkat**

`allow_origins=["*"]` yazmak (herkese izin vermek) kolay bir çözüm gibi görünse de kötü bir alışkanlıktır — üretim ortamında güvenlik riski taşır (ayrıntı Hf.11). Bu ders boyunca `allow_origins` listesi her zaman belirli, bilinen adresleri içerir.

### 7.3.5 DevTools Network sekmesinde preflight'ı gözlemleme

Sunucu çalışırken bir `POST /biletler` isteği tetiklendiğinde, Network sekmesinde ayrı bir `OPTIONS /biletler` satırının ve hemen ardından gerçek `POST /biletler` satırının görülmesi gerekir. `curl` ile bu ön uçuş isteğini elle de tetikleyip cevap başlıklarını görebilirsiniz:

Terminal

```
$ curl -s -i -X OPTIONS http://127.0.0.1:8000/biletler \
  -H "Origin: http://localhost:5173" \
  -H "Access-Control-Request-Method: POST" \
  -H "Access-Control-Request-Headers: Content-Type"
```

Beklenen Çıktı

```
HTTP/1.1 200 OK
access-control-allow-methods: DELETE, GET, HEAD, OPTIONS, PATCH, POST,
↪  PUT
access-control-max-age: 600
access-control-allow-credentials: true
access-control-allow-origin: http://localhost:5173
access-control-allow-headers: Content-Type
```

Cevap başlıklarında `access-control-allow-origin` ve `access-control-allow-methods`'un varlığına dikkat edin — bunlar tarayıcının asıl `POST`'u göndermesi için gereken "izin kanıtıdır". `Origin` başlığı izinli listede olmayan bir adresle (`localhost:4000`) gönderilirse bu başlıklar cevapta hiç görünmez — iki testin de doğruladığı budur:

```python
def test_cors_izinli_origin(client):
    cevap = client.get(
        "/etkinlikler", headers={"Origin": "http://localhost:5173"}
    )
    assert cevap.status_code == 200
    assert cevap.headers["access-control-allow-origin"] == "http://localhost
        :5173"
```

```python
def test_cors_izinsiz_origin(client):
```

```
    cevap = client.get(
        "/etkinlikler", headers={"Origin": "http://localhost:4000"}
    )
    assert cevap.status_code == 200
    assert "access-control-allow-origin" not in cevap.headers
```

İkinci testin `cevap.status_code == 200` denetlediğine dikkat edin — istek sunucuda yine de çalışır; CORS bir sunucu yetkilendirmesi değil, tarayıcının cevabı saklamasını sağlayan bir başlık mekanizmasıdır, izinsiz köken'de yalnızca bu başlık eksik olur.

**İpucu**

Vite'ın `server.proxy` seçeneği, geliştirme sunucusunun `/api` gibi bir ön ekle başlayan istekleri otomatik olarak `8000`'e yönlendirebilir (bu durumda tarayıcı isteği kendi köken'ine atmış gibi görür, CORS hiç devreye girmez). Bu yaklaşım yalnızca geliştirme ortamında işe yarar — üretimde yine gerçek bir CORS/ters vekil yapılandırması gerekir. Bu kitapta CORS'un kendisini öğretmek amaçlandığı için bu ders bu yolu kullanmaz.

**3 • Katılım — Şimdi Siz Deneyin**

`CORSMiddleware`'i eklemeden önce tarayıcı konsolunda bir `fetch` denemesi yapıp hata mesajını okuyun; Network sekmesinde isteğin durumunu inceleyin. Sonra `CORSMiddleware`'i ekleyip sunucuyu yeniden başlatın, aynı isteği tekrar deneyin, Network sekmesinde yeni beliren `OPTIONS` isteğinin cevap başlıklarını inceleyin.

---

**Geribildirim.** eklemeden önce konsolda "has been blocked by CORS policy" benzeri bir mesaj görünür, veri ekrana hiç gelmez. Ekledikten sonra bir `OPTIONS` isteği (durum `200`) ve hemen ardından asıl istek görünür; `access-control-allow-origin: http://localhost:5173` başlığı cevapta yer alır, veri ekrana gelir.

**Kendinizi Sınayın 7.3**

Vite (5173) üzerinden çalışan bir React uygulaması FastAPI'ye (8000) JSON gövdeli bir `POST /biletler` isteği gönderdiğinde tarayıcı ilk olarak hangi isteği gönderir, neden?

**5 • İzleme — Aktarım — Başka Nerede?**

`allow_origins` listesi Hf.14'te gerçek üretim alan adınıza güncellenecek; Hf.11 CORS'u bir güvenlik konusu olarak (`allow_origins=["*"]` + kimlik bilgisi riskleri) daha derin ele alacak — bugün öğrendiğiniz preflight/başlık mekanizması değişmeyecek, yalnızca yapılandırması sıkılaşacak.

## 7.4 React Router: Sayfalar, Parametreli Rotalar

**1 • Ön-öğretim — Neden Önemli?**

Şu ana kadar tek bir "sayfanız" vardı — adres çubuğu hiçbir zaman değişmedi, bir etkinliğe doğrudan bir bağlantı paylaşamadınız, tarayıcının geri düğmesi işe yaramadı. Gerçek bir web sitesinin her anlamlı görünümü kendi URL'sine sahiptir.

**1 • Ön-öğretim — Ön Bilgi**

Hf.1 Ünite 3'ün SPA vs SSR ayrımını hatırlayın — bu ünite bu ayrımın ilk çalışan kanıtını sunar. Hf.4'ün bileşen ağacı bilgisini de hatırlayın — sayfalar da birer bileşendir, farkları yalnızca bir `Route`'a bağlanmalarıdır.

Ünite 1-3 sunucudan gerçek veri çekmeyi ve CORS engelini aşmayı öğretti. Bu son ünite, Hf.4-6 boyunca tek bir `App` bileşeninin içinde yaşayan uygulamanızı, kendi URL'sine sahip dört ayrı "sayfaya" böler: etkinlik listesi, etkinlik detayı, biletlerim ve 404. Bu bölümün sonunda Etkinlik/Bilet Sisteminin ön yüzü, gerçek sunucudan veri çeken, koltuk çakışmasını (`409`) kullanıcıya gösteren, çok sayfalı bir SPA'ya dönüşmüş olacak.

### 7.4.1 `react-router`: paket ve API seçimi

Bu ders `BrowserRouter` + `Routes` + `Route` + `Link` + `useParams` + `useNavigate` + `Outlet` bildirimsel (*declarative*) API'sini kullanan tek bir paket kurar: `react-router`. react-router projesi v7 ile `react-router` ve `react-router-dom` paketlerini birleştirdi — `react-router` paketinin kendisi artık web için gereken tüm DOM bağlamalarını doğrudan içerir; bu ders yeni bir `react-router-dom` bağımlılığı kurmaz. Router'ın kendi veri çekme katmanı (`loader`, `createBrowserRouter`) bu haftanın kapsamı dışındadır — Ünite 1-2'nin `useVeri` hook'unu bileşen içinde çağırma yaklaşımıyla yetinirsiniz.

**Dikkat**

İki farklı veri çekme felsefesini (router'ın kendi veri katmanı ile bileşenin kendi `useEffect`'i) aynı haftada göstermek bilişsel yükü gereksiz artırır — bu ders bilinçli olarak ikincisini (Ünite 1-2'de kurduğunuz `useVeri`) seçer.

### 7.4.2 `BrowserRouter`, `Routes`, `Route`

`<BrowserRouter>`, uygulamanın tamamını saran, tarayıcının `history` API'sini kullanan kök bileşendir — `App.tsx`'in en dış katmanıdır. İçinde `<Routes>` yer alır; `Routes` içindeki `Route`'lar bir URL yolunu bir bileşenle eşler ve sırayla değerlendirilir — ilk eşleşen yol render edilir.

**Analoji**

Bunu bir alışveriş merkezinin girişindeki kat yönlendirme tabelasına benzetin: tabelada "1. Kat → Giyim", "2. Kat → Elektronik" yazar; hangi kata gitmek istediğinizi (URL) söylediğinizde doğru mağazaya (bileşene) yönlendirilirsiniz.

### 7.4.3 `Link`: sayfa yenilemeden gezinme

`<Link to="...">`, normal bir `<a href="...">` etiketinin React Router karşılığıdır — `<a>`'nın aksine tıklandığında tarayıcıya yeni bir HTTP isteği göndertmez, yalnızca adres çubuğunu güncelleyip eşleşen bileşeni render eder. Gezinme tarayıcıdan bağımsız, JavaScript içinde gerçekleşir — Hf.1 Ünite 3'ün SPA tanımının somut kanıtıdır.

**Analoji**

`Link` ile gezinmeyi bir binanın içindeki asansörle kat değiştirmeye benzetin: binadan hiç çıkmadan (sunucuya yeni istek gitmeden) bir kattan diğerine geçersiniz; `<a>` etiketiyle dış bağlantıya tıklamak ise binadan çıkıp başka bir binaya (yeni bir HTTP isteğiyle) gitmek gibidir.

### 7.4.4 Parametreli rota ve `useParams`

**parametreli rota**: Yol şablonunda değişken bir bölüm (`:id`) içeren rota tanımı (*dynamic route*).

Aşağıdaki rota tanımındaki `:id`, URL'nin değişken bir parçasını yakalayan bir **parametreli rota**dir — Hf.6'nın yol parametresi `{etkinlik_id}` kavramının ön yüz karşılığıdır:

```
<Route path="/etkinlikler/:id" element={<EtkinlikDetaySayfasi />} />
```

`EtkinlikDetaySayfasi` içinde bu değere `useParams` ile erişilir:

```
const { id } = useParams<{ id: string }>();
const etkinlikId = Number(id);
```

**Ortak Hata / Tuzak**

`id` Her zaman bir `string`'dir — URL bir metindir, sayı değildir. `apiGet` çağrısına `id`'yi doğrudan, `Number(id)` ile çevirmeden geçirmek bir tip hatasına (TypeScript `number` beklerken `string` aldığı için) ya da çalışma zamanında yanlış bir URL oluşturulmasına yol açar.

**Analoji**

Parametreli rotayı bir apartman dairesinin kapı numarasına benzetin: binanın (rota şablonunun) yapısı hep aynıdır ("`/etkinlikler/`" koridoru), yalnızca kapı numarası (`:id`) her dairede değişir.

### 7.4.5 `useNavigate`: koddan tetiklenen gezinme

`Link`'in aksine bir olay işleyicisi içinden (ör. bilet formu başarıyla gönderildikten sonra) programatik olarak başka bir sayfaya geçmek için `useNavigate` kullanılır — bir kullanıcı tıklaması gerektirmez, kod içinden tetiklenir:

```
const navigate = useNavigate();
navigate(`/biletlerim?kullaniciAdi=${encodeURIComponent(kullaniciAdi)}`);
```

Bu, Ünite 2'nin "fikir düzeyinde" bıraktığı iyimser güncelleme deseni ile birlikte, `BiletFormu`'nun gönderim mantığında gerçek koduyla karşınıza çıkar — doğrulama (ad boş mu, sepette hiç koltuk seçili mi değil mi) Hf.2'deki HTML formuyla aynı kalır:

```
async function submitEt(e: FormEvent<HTMLFormElement>) {
  e.preventDefault();

  if (form.ad.trim() === '') {
    setHata('Ad alanı boş olamaz.');
    return;
  }
  if (secilenKoltuklar.length === 0) {
    setHata('Sepette en az bir koltuk seçili olmalı.');
    return;
  }
```

Doğrulama geçtiğinde her seçili koltuk için gerçek bir `POST /biletler` isteği (`biletOlustur`) `Promise.all` ile paralel gönderilir; başarıda sepet temizlenir, form sıfırlanır ve `useNavigate` ile `/biletlerim`'e yönlendirilirsiniz:

```
try {
  const yeniBiletler = await Promise.all(
    secilenKoltuklar.map((koltuk) =>
      biletOlustur({ koltukId: koltuk.id, kullaniciAdi: form.ad }),
    ),
  );

  onBiletlerOlustu(yeniBiletler);
  sepetiTemizle();
  setHata(null);
  const kullaniciAdi = form.ad;
  setForm(BOS_FORM);
  navigate(`/biletlerim?kullaniciAdi=${encodeURIComponent(kullaniciAdi)}`);
```

Sunucu bu arada koltuğu başkasına satmışsa `409` döner — apiPost bunu bir `ApiHatasi` olarak fırlatır, burada yakalanıp kullanıcıya gösterilir (Hf.6'da yalnızca `curl` ile gözlenen bu iş kuralı artık gerçek arayüze yansır):

```
if (e instanceof ApiHatasi && e.status === 409) {
  setHata('Seçtiğiniz koltuklardan biri bu arada başkasına satıldı.');
} else {
  setHata('Bilet oluşturulurken bir hata oluştu. Lütfen tekrar deneyin.');
}
```

### 7.4.6 Hepsi bir arada: `EtkinlikDetaySayfasi`

Bu haftanın MERKEZÎ sayfası `EtkinlikDetaySayfasi`'dır — `useParams` ile URL'den gelen `id`'yi okur, `useVeri` ile hem etkinliği hem koltukları tek bir çağrıda çeker, Ünite 2'nin fikir düzeyinde bıraktığı iyimser koltuk seçimini gerçek koduyla uygular ve `KoltukPlani`/`SepetOzeti`/`GeriSayim`/`BiletFormu` bileşenlerini tek bir sayfada birleştirir. Aşağıdaki üç kod parçası bu sayfanın veri çekme, koltuk seçim ve render katmanlarını sırayla gösterir.

**Veri çekme: etkinlik ve koltuklar birlikte.** İlk parça, `useParams`'tan gelen `id`'yi `Number` ile sayıya çevirip `useVeri` hook'una bağımlılık olarak verir — `etkinlikId` değiştiğinde (kullanıcı başka bir etkinliğe tıkladığında) efekt yeniden çalışır, Ünite 2'nin `AbortController`'ı önceki isteği iptal eder. `Promise.all` ile etkinlik ve koltuk verisi tek bir çağrıda, paralel olarak çekilir — iki ayrı `useVeri` çağrısı yerine tek bir birleşik durum (`EtkinlikDetayVerisi`) kullanılması, yükleniyor/hata durumlarının tek bir yerde yönetilmesini sağlar:

```
export function EtkinlikDetaySayfasi() {
  const { id } = useParams<{ id: string }>();
```

```
  const etkinlikId = Number(id);

  const durum = useVeri<EtkinlikDetayVerisi>(
    async (signal) => {
      const [etkinlik, koltuklar] = await Promise.all([
        etkinlikGetir(etkinlikId, signal),
        koltuklariGetir(etkinlikId, undefined, signal),
      ]);
      return { etkinlik, koltuklar };
    },
    [etkinlikId],
  );
```

Sayfa `yukleniyor`/`hata` dallarını Ünite 2'nin `Yukleniyor`/`HataMesaji` bileşenleriyle aynen ele alır; başarı durumunda ise etkinliğin `biletFiyati`'nı `SepetProvider`'a (Hf.5) geçirip alt bileşeni sarar:

```
  if (durum.tur === 'yukleniyor') {
    return <Yukleniyor />;
  }
  if (durum.tur === 'hata') {
    return <HataMesaji mesaj={durum.mesaj} />;
  }

  return (
    <SepetProvider biletFiyati={durum.veri.etkinlik.biletFiyati}>
      <EtkinlikDetayIcerik
        etkinlik={durum.veri.etkinlik}
        ilkKoltuklar={durum.veri.koltuklar}
      />
    </SepetProvider>
  );
}
```

Context'i sağlayan bileşenin kendisi o Context'i okuyamaz — bu yüzden `SepetProvider`'ın içinde, Hf.5'in `AppIcerik` desenindeki gibi ayrı bir `EtkinlikDetayIcerik` bileşeni gerekir.

**İyimser koltuk seçimi: fikirden koda.** İkinci parça, Ünite 2'nin yalnızca fikir düzeyinde bıraktığı iyimser güncellemeyi gerçek koduyla gösterir. `useSepet`'in döndürdüğü `secilenKoltuklar` dizisi her değiştiğinde (kullanıcı bir koltuğa tıkladığında) bir `useEffect` çalışır; bu efekt yeni eklenen koltuğu `izlenenIdler` ref'i ile öncekilerden ayırt eder:

```
useEffect(() => {
  const guncelIdler = new Set(secilenKoltuklar.map((k) => k.id));
  const yeniSecilen = secilenKoltuklar.find(
    (k) => !izlenenIdler.current.has(k.id),
  );
  izlenenIdler.current = guncelIdler;

  if (!yeniSecilen) return;
```

Yeni bir koltuk tespit edildiğinde o koltuk için `koltukGuncelle` ile bir `PATCH` isteği gönderilir. Kullanıcı arayüzde koltuğu anında "seçili" görür — `koltukSec` çağrısı sunucu cevabını beklemeden state'i güncellemiştir; sunucu `409` dönerse (koltuk bu arada başkası tarafından alınmışsa) `koltukSec` tekrar çağrılarak seçim geri alınır ve yerel koltuk listesi "dolu" olarak güncellenir. Bu, garson analojisindeki "fiş geri alınması" anına tam olarak karşılık gelir:

```
  koltukGuncelle(yeniSecilen.id, 'secili')
    .then(() => setKoltukHatasi(null))
    .catch((hata) => {
      koltukSec(yeniSecilen);
      if (hata instanceof ApiHatasi && hata.status === 409) {
        setKoltukHatasi('Bu koltuk bu arada başka bir kullanıcı tarafından
            alındı.');
        setKoltuklar((onceki) =>
          onceki.map((k) =>
            k.id === yeniSecilen.id ? { ...k, durum: 'dolu' } : k,
          ),
        );
      } else {
        setKoltukHatasi('Koltuk güncellenirken bir hata oluştu.');
      }
    });
}, [secilenKoltuklar]);
```

**Ortak Hata / Tuzak**

`izlenenIdler` Ref'i olmadan, efekt `secilenKoltuklar` dizisindeki her elemanı yeniden sunucuya gönderirdi — ref, hangi koltuğun yeni eklendiğini hatırlayarak yalnızca tek bir `PATCH` isteği atılmasını sağlar.

**Render: sayfanın tamamı.** Üçüncü ve son parça, sayfanın döndürdüğü JSX'i gösterir — etkinlik başlığı, `KoltukPlani` (koltuk hatası varsa `HataMesaji` ile birlikte), `SepetOzeti`, `GeriSayim` ve `BiletFormu` ayrı `<section>` etiketleri içinde alt alta dizilir; sayfanın tam JSX'i `kod_ornekleri/hafta07/istemci/src/sayfalar/EtkinlikDetaySayfasi.tsx` dosyasındadır. Bu üçlemenin gerçekten yeni olan parçası `BiletFormu`'nun `onBiletlerOlustu` geri çağrısıdır — bir üstteki `useNavigate` alt başlığında gördüğünüz gönderim mantığıyla birlikte çalışan bu geri çağrı, oluşturulan biletleri yerel bir diziye ekleyip sayfanın altında kısa bir özet gösterir:

```
<BiletFormu
  onBiletlerOlustu={(yeniBiletler) =>
    setBiletler((onceki) => [...onceki, ...yeniBiletler])
  }
/>
```

Bu üç parça birlikte, Ünite 1'in `fetch`/tiplenmiş istemcisini, Ünite 2'nin yükleniyor/hata/iyimser güncelleme desenlerini ve bu ünitenin `useParams`/`Outlet` altyapısını tek bir sayfada birleştirir — Etkinlik/Bilet Sisteminin ön yüzünün en karmaşık, ama en temsili sayfasıdır.

### 7.4.7 `Outlet` ve ortak sayfa düzeni

**sayfa düzeni**: Alt sayfaların ortak çerçevesini (nav, dipnot) taşıyan sarmal bileşen (*layout*).

Dört sayfanın (etkinlik listesi, etkinlik detayı, biletlerim, 404) her birine aynı navigasyon çubuğunu tekrar tekrar yazmak yerine, bu ünite alt sayfaların ortak çerçevesini (nav, dipnot) taşıyan bir **sayfa düzeni** kurar. `Duzen` adını taşıyan bu bileşen bir navigasyon çubuğu içerir, içinde bir `<Outlet />` bulundurur — saran `Route` içine yerleştirilen tüm iç sayfalar bu tek navigasyon çubuğunu paylaşır. Bu sarmalayıcı olmadan her sayfa kendi navigasyon kodunu ayrı ayrı yazardı, bu da hem tekrar hem de sayfalar arası tutarsızlık riski doğururdu. Aşağıdaki kod `Duzen`'in tam tanımını gösterir:

```
export function Duzen() {
```

```
  return (
    <div className="duzen">
      <nav className="duzen-nav">
        <span className="duzen-baslik">KTÜ Etkinlik ve Bilet Sistemi</span>
        <Link to="/">Etkinlikler</Link>
        <Link to="/biletlerim">Biletlerim</Link>
      </nav>
      <main>
        <Outlet />
      </main>
    </div>
  );
}
```

**Outlet**, o andaki eşleşen alt rota bileşeninin (etkinlik listesi, etkinlik detayı ya da biletlerim sayfası) render edileceği yer tutucudur. `Duzen`'i saran `Route` içine yerleştirilen iç `Route`'lar hep aynı navigasyon çubuğunun altında görünür.

**Analoji**

Layout/`Outlet`'i bir derginin her sayfasında aynı kalan başlık/dipnot şeridine benzetin: üstteki logo ve alttaki sayfa numarası (nav çubuğu) hiç değişmez, yalnızca ortadaki makale içeriği (Outlet) sayfadan sayfaya değişir.

### 7.4.8 404 sayfası: eşleşmeyen Her yol için

`<Route path="*" element={<BulunamadiSayfasi />} />`, `Routes` içindeki hiçbir yolla eşleşmeyen herhangi bir URL için en sonda tanımlanan özel bir "joker" (`*`) yoldur:

**Analoji**

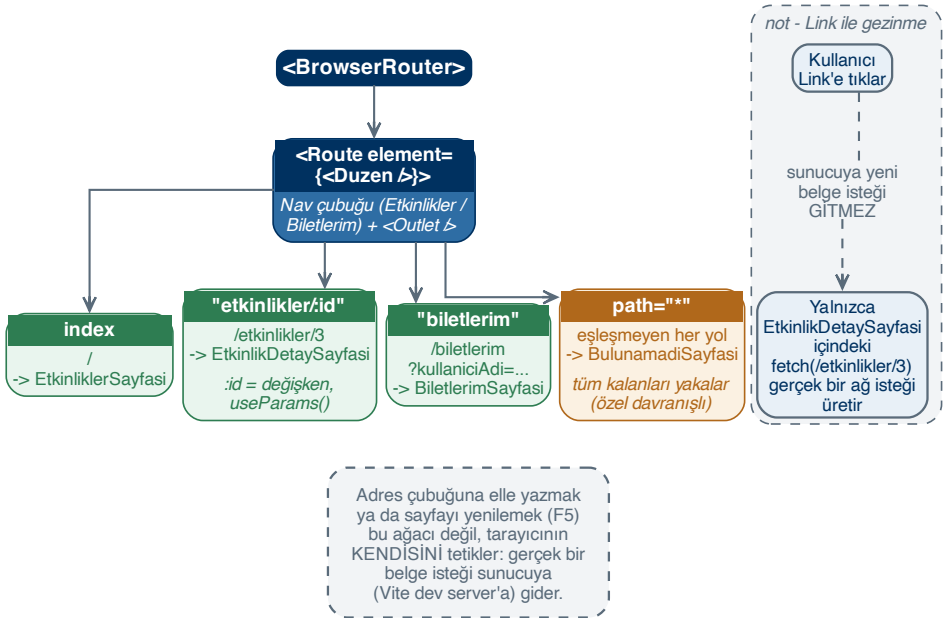
Bir 404 sayfasını, elinizdeki bir adrese gidip o adreste hiçbir binanın olmadığını, boş bir arsa gördüğünüzü fark etmenize benzetin: adres (URL) yazım olarak geçerlidir, ama orada tanımlı bir "yer" (rota) yoktur.

### 7.4.9 Tüm rotaların birleştiği yer: `App.tsx`

Yukarıdaki tüm parçalar `App.tsx`'te bir araya gelir:

```
export function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route element={<Duzen />}>
          <Route index element={<EtkinliklerSayfasi />} />
          <Route path="etkinlikler/:id" element={<EtkinlikDetaySayfasi />} />
          <Route path="biletlerim" element={<BiletlerimSayfasi />} />
          <Route path="*" element={<BulunamadiSayfasi />} />
        </Route>
      </Routes>
    </BrowserRouter>
  );
}
```

Şekil 7.4 bu yapıyı bir ağaç olarak gösterir — kökten `Duzen`'e, oradan dört iç rotaya dallanma ve `Link` ile gezinmede sunucuya yeni istek gitmediği.




Şekil 7.4: URL → `Route` eşlemesi, `Duzen/Outlet` konumu ve `Link` ile gezinmede sunucuya yeni istek gitmediği.

### 7.4.10 Sayfa yenileme (F5) ile `Link` tıklaması aynı şey değildir

Adres çubuğuna doğrudan `/etkinlikler/5` yazıp Enter'a basmak ya da tarayıcıyı yenilemek (F5) bir `Link` tıklaması değildir — tarayıcı gerçek bir HTTP isteği gönderir. Vite'ın geliştirme sunucusu bu isteği yakalayıp `index.html`'e düşürecek şekilde önceden yapılandırılmıştır, bu yüzden bu hafta hiçbir sorun yaşanmaz.

> **Dikkat**
>
> Gerçek bir statik dosya sunucusunda (üretim ortamı) bu yolu bir dosya gibi arayıp bulamayıp `404` dönebilir — bu sorunun çözümü (sunucunun tüm bilinmeyen yolları `index.html`'e yönlendirmesi) Hf.14'ün dağıtım konusudur; bu hafta yalnızca sorunun varlığı açıkça belirtilir, çözülmez.

### 7.4.11 Doğrulama: kapı komutları

Bu haftanın tüm istemci değişikliklerini iki komutla doğrularsınız:

Terminal

```
$ npx tsc -b
$ npm run build
```

Beklenen Çıktı

```
vite v8.2.2 building client environment for production...
transforming...
✓ 97 modules transformed.
rendering chunks...
computing gzip size...
✓ built in 81ms
```

Sunucu tarafında Hf.6'nın 9 testi ile bu haftanın 2 yeni CORS testi birlikte çalışır:

Terminal

```
$ python -m pytest -q
```

Beklenen Çıktı

```
...........                                                  [100%]
11 passed in 0.05s
```

**3 · Katılım** **Şimdi Siz Deneyin**

Bir `EtkinlikKarti`'ndeki `Link`e tıklayıp DevTools Network sekmesinde yeni bir HTML/JS dosyası indirilmediğini (yalnızca `fetch` isteklerinin göründüğünü) doğrulayın; sonra aynı URL'i adres çubuğuna elle yazıp Enter'a basarak sayfayı tam yenileyin, Network sekmesindeki farkı gözlemleyin.

**Geribildirim.** `Link` tıklaması sırasında Network sekmesinde yalnızca `fetch` istekleri görünür, hiçbir yeni belge indirilmez (React Router JS içinde geçiş yapar). Adres çubuğuna elle yazıp Enter'a basmak gerçek bir belge isteği üretir — tarayıcı tüm uygulamayı baştan yükler.

**Kendinizi Sınayın 7.4**

`/etkinlikler/5` sayfasındayken tarayıcıyı yenilerseniz (F5) bu isteği React Router mı, yoksa sunucu mı karşılar — ve bu neden Hf.14'e kadar bir sorun oluşturmaz?

**5 · İzleme** **Aktarım — Başka Nerede?**

Bugün kurduğunuz `Duzen`+`Outlet`+sayfa iskeleti, Hf.10'da `/giris`, `/kayit` ve korumalı `/panel` rotalarıyla; Hf.15'te özellik-tabanlı klasörlemeyle (her sayfa kendi klasöründe: bileşen+ hook+api) genişleyecek. Bugünkü birleşimle Etkinlik/Bilet Sisteminin ön yüzü artık Hf.6'nın gerçek sunucusuna bağlı, yükleme/hata durumlarını yöneten, CORS engelini aşan ve kendi URL'sine sahip dört sayfadan oluşan çok sayfalı bir SPA'dır.

## Bölüm Özeti

**Özet**

Bu hafta Hf.1-5'in yalnızca tarayıcıda yaşayan ön yüzü ile Hf.6'nın yalnızca kendi başına çalışan sunucusunu birleştirdiniz. İlk ünitede `fetch`'in bir Promise döndürdüğünü, bu Promise'in yalnızca bir ağ hatasında reddedildiğini — sunucunun herhangi bir HTTP cevabının (404 dahil) Promise'i normal şekilde çözdüğünü — gördünüz ve bu ayrımı tek bir yerde toplayan `apiGet`/`apiPost` istemcisini yazdınız. İkinci ünitede `yukleniyor | hata | basarili` üç durumlu modelini bir ayrımcı birleşim tipiyle tanımlayıp `useVeri<T>` özel hook'unda

uyguladınız; `AbortController` ile yarış durumunu nasıl önleyeceğinizi gördünüz. Üçüncü ünitede tarayıcının aynı-köken politikasını, ön uçuş `OPTIONS` isteğini ve `Access-Control-Allow-Origin` başlığını öğrenip FastAPI'ye tek bir `CORSMiddleware` bloğu eklediniz. Dördüncü ünitede `react-router` ile uygulamanızı `Duzen`+`Outlet` ortak düzeni altında dört ayrı sayfaya böldünüz — parametreli rotalar, `Link` ile sayfa yenilemeden gezinme ve bir 404 sayfası dahil. Sonuçta Etkinlik/Bilet Sisteminin ön yüzü nihayet Hf.6'nın gerçek sunucusuna bağlandı — koltuk çakışması artık `curl` ile değil, gerçek arayüzde `409` olarak görünüyor. Gelecek hafta bu sunucunun hâlâ geçici olan belleği, SQLAlchemy ile gerçek bir veritabanına (SQLite) taşınarak kalıcı hâle gelecek.

## Kendinizi Sınayın Cevapları

**Soru (Ünite 1 — 404 ile Promise davranışı).** Sunucudan `'/etkinlikler/999'` yoluna bir istek yapıp `404` cevabı döndüğünde, bu isteğin Promise'i normal şekilde çözülür (resolve) — reddedilmez (reject). `fetch` Yalnızca bir ağ hatasında (DNS çözülemedi, bağlantı hiç kurulamadı, CORS tamamen engelledi) reddedilir; sunucunun herhangi bir HTTP cevabı (`404` dahil) başarılı bir ağ iletişimi sayılır — sunucu bir cevap üretmiştir. Bu durumda `response.ok` `false` olur (durum `200-299` aralığı dışında), `response.status` `404`'tür. Bu yüzden bu durum `try/catch` ile değil, `response.ok` kontrolüyle yakalanmalıdır — Ünite 1'in `apiGet` fonksiyonu bu kontrolü sizin için yapıp bir `ApiHatasi` fırlatır.

**Soru (Ünite 2 — `AbortController` olmadan yarış durumu).** `useVeri` hook'unda `AbortController` kullanmasaydınız, kullanıcı `/etkinlikler/3`'ten `/etkinlikler/7`'ye hızlıca geçtiğinde iki ayrı istek yola çıkardı. Eğer 3 numaralı etkinliğin isteği ağ gecikmesi yüzünden 7 numaralınınkinden daha geç cevap verirse, bu geç gelen cevap state'i güncelleyip kullanıcı artık 7 numaralı sayfadayken ekranda 3 numaralı etkinliğin verisini gösterebilirdi — "eski cevap yeni cevabı ezer". `AbortController` önceki isteği iptal ederek bu eski cevabın state'i güncellemesini engeller; `AbortError` fırlatıldığında `useVeri` bu hatayı sessizce görmezden gelir.

**Soru (Ünite 3 — ilk gönderilen istek ve nedeni).** Vite (5173) üzerinden çalışan bir React uygulaması FastAPI'ye (8000) JSON gövdeli bir `POST /biletler` isteği gönderdiğinde tarayıcı ilk olarak bir ön uçuş `OPTIONS /biletler` isteği gönderir — çünkü JSON gövdeli bir `POST` "basit olmayan" bir istek sayılır. Tarayıcı asıl isteği göndermeden önce sunucuya "bu köken'e (5173), bu metotla, bu başlıklarla istek atabilir miyim?" diye sorar; sunucu uygun `Access-Control-Allow-*` başlıklarıyla cevap verirse tarayıcı ancak O zaman asıl `POST` isteğini gönderir — bu iki ayrı ağ isteği, DevTools Network sekmesinde iki ayrı satır olarak görünür.

**Soru (Ünite 4 — F5 yenilemesini kim karşılar).** `/etkinlikler/5` sayfasındayken tarayıcıyı yenilerseniz (F5), bu isteği React Router değil, Vite'ın geliştirme sunucusu karşılar. F5 tarayıcının gerçek, yeni bir HTTP isteği göndermesine yol açar — bu, `Link` tıklamasıyla yapılan bir SPA-içi gezinme değildir; React Router'ın JavaScript içindeki eşleme mekanizması bu isteğin içine hiç girmez. Bu neden Hf.14'e kadar bir sorun oluşturmaz: Vite'ın geliştirme sunucusu bu isteği zaten `index.html`'e düşürecek şekilde önceden yapılandırılmıştır. Gerçek bir üretim ortamında basit bir

statik dosya sunucusu bu yolu bir dosya sanıp bulamayabilir ve `404` dönebilir — bu sorunun çözümü (sunucunun tüm bilinmeyen yolları `index.html`'e yönlendirmesi) Hf.14'ün dağıtım konusudur.